
React Components

Objectives

- What a component is
 - How to create a React component
 - React Components: PROPs and STATE
 - React Components: List & Keys
 - Presentational & Container Component
-

What a component is

React Component

- Components are **one of the core concepts** of React. They are the foundation upon which you build user interfaces (UI).
- A React component is a JavaScript function that you can sprinkle with markup. React lets you combine your markup, CSS, and JavaScript into custom “components”, reusable UI elements for your app.
- Conceptually, components are like JavaScript functions. They accept arbitrary inputs (called “props”) and return React elements describing what should appear on the screen.
- React components are regular JavaScript functions, but their names **must start** with a **capital letter** or they won't work!

React Components: LifeCycle Methods

- React Component Lifecycle is the set of phases that a component goes through from creation, updating, to unmounting.
- Understanding the lifecycle helps you intervene at the right time to perform necessary tasks (e.g., calling APIs, updating state, cleaning up resources, optimizing performance).
- The lifecycle is especially important for **Class Components**, but for **Function Components**, we use Hooks to handle similar processes.

React Component Lifecycle (Class Components)

Phase	Main Meaning	Some Typical Lifecycle Methods
Mounting	When the component is created and inserted into the DOM	constructor(), render(), componentDidMount()
Updating	When the component receives new props or state changes and re-renders	shouldComponentUpdate(), render(), componentDidUpdate()
Unmounting	When the component is removed from the DOM	componentWillUnmount()
Error Handling	When an error occurs during rendering or lifecycle	componentDidCatch()

React Component

➤ Example:

```
1 export default function Profile() {  
2   return (  
3       
7   )  
8 }  
9
```



Demo & Some challenges

- Demo – Defining a componnet

Demo & Some challenges

- Some challenges

➤ Challenge 1:

```
1 function Profile() {  
2   return (  
3       
7   );  
8 }  
9
```

Error

Element type is invalid: expected a string (for built-in components) or a class/function (for composite components) but got: object. You likely forgot to export your component from the file it's defined in, or you might have mixed up default and named imports.

Demo & Some challenges

- Some challenges

➤ **Challenge 2:**

```
1 export default function Profile() {  
2   return  
3       
7   );  
8 }  
9  
10 export default function Gallery() {  
11   return (  
12     <section>  
13       <h1>Hello, FER202!</h1>  
14       <profile />  
15       <profile />  
16       <profile />  
17     </section>  
18   );  
19 }  
20
```

Hello, FER202!

➤ Console (1)



React Components: PROPs and STATE

PROPS

- React components use **props** to **communicate** with each other.
- Every parent component can pass some information to its child components by giving them props.
- Props might remind you of HTML attributes, but you can pass any JavaScript value through them, including objects, arrays, and functions.
- Props are the information that you pass to a **JSX tag**.
For example: **className**, **src**, **alt**, **width**, and **height** are some of the props you can pass to an `<img>`
- **Cannot modify props** within the component. Props are **Read-Only**.

PROPS

You will learn:

- How to pass props to a component
- How to read props from a component
- How to specify default values for props
- Forwarding props with the JSX spread syntax

STATE

- **State** is a **Component's Memory**.
- Components often need to change what's on the screen as a result of an interaction. Components need to “remember” things: the current input value, the current image, the shopping cart. In React, this kind of component-specific memory is called **state**, etc.
- Each component can store its own local information in its “state”.
 - Private and fully controlled by the component
 - Can be passed as props to children

React Components: LISTS and KEYs

Rendering Lists

Using `map()` and `filter()` with React to filter and transform your array of data into an array of components.

You will learn:

- Rendering data from arrays
- Filtering arrays of items
- Keeping list items in order with key

Rendering Lists

Rendering lists example

```
1  const people = [  
2    'Creola Katherine Johnson: mathematician',  
3    'Mario José Molina-Pasquel Henríquez: chemist',  
4    'Mohammad Abdus Salam: physicist',  
5    'Percy Lavon Julian: chemist',  
6    'Subrahmanyam Chandrasekhar: astrophysicist'  
7  ];  
8  
9  export default function List() {  
10    const listItems = people.map(person =>  
11      <li>{person}</li>  
12    );  
13    return <ul>{listItems}</ul>;  
14  }  
15
```

- Creola Katherine Johnson: mathematician
- Mario José Molina-Pasquel Henríquez: chemist
- Mohammad Abdus Salam: physicist
- Percy Lavon Julian: chemist
- Subrahmanyam Chandrasekhar: astrophysicist

React Component Types:

Presentation and Container Components

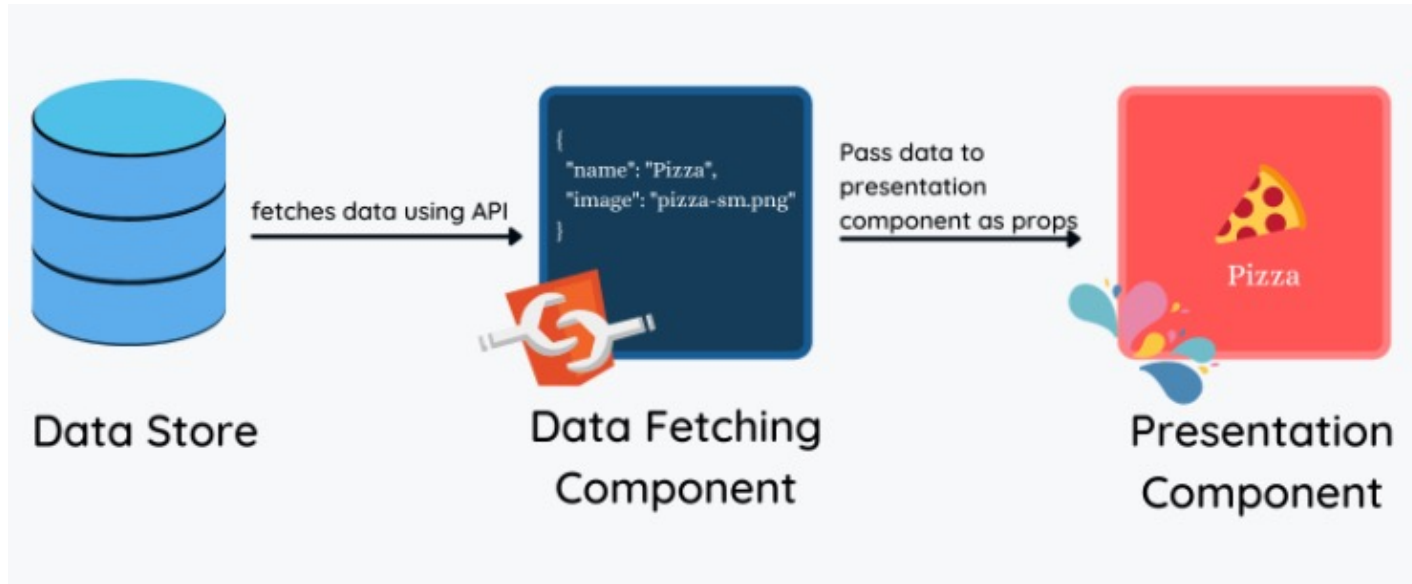
Presentational Components

- Are components responsible for what appears on the screen that the user can see.
- The main functions of presentational components are:
 - Responsible for **how everything is displayed** (HTML, CSS).
 - Receive data and callbacks via props.
 - Do not specify how data is loaded or modified.
 - Usually **stateless components** (do not have internal state) unless state is needed for handling UI parts.
 - Do not contain other dependencies within the application.

Container Components

- These are components responsible for data handling. For example, handling image fetching from the server.
- Some of the main functions of container components are:
 - Responsible for **how everything works**.
 - Provide data and behavior to presentational or other container components.
 - API calls as callbacks to presentational components.
 - Usually **stateful** and serve as the source of data.

Container Components



Exercises

Summary

- How to create a React component
 - Use JSX to define the views of your component
 - React State and Props to passing data between components
 - React components types: Presentational and Container Components
 - Create presentational, container components for react app
-